Spamdeteksjon med ML

Sander Holen, 30.10.2026

1: BESKRIV PROBLEMET

OMFANG / SCOPE

Spam er noe de fleste som bruker digitale kommunikasjonskanaler har opplevd. Ofte inneholder slike meldinger reklame man ikke har bedt om, forsøk på svindel eller lenker til nettsider som kan være skadelige. Siden det daglig kan komme mange meldinger, tar det både tid og kan være utfordrende for brukere å skille mellom vanlige meldinger og spam.

Jeg ønsker derfor å lage en enkel nettside som bruker maskinlæring til å avgjøre om en SMS-melding er spam eller ikke. Tanken er at brukeren skal kunne lime inn en melding på nettsiden, og så får de et svar fra maskinlæringsmodellen på om meldingen er spam eller ikke.

Maskinlæring passer godt til denne typen oppgave fordi modellen kan lære seg mønstre fra tidligere eksempler på meldinger. I stedet for å lage faste regler for hvilke ord som er spam, kan en maskinlæringsmodell finne ut hvilke tekstmønstre som kjennetegner slike meldinger.

Det finnes lignende løsninger i e-posttjenester og meldingsapper, der automatiske spamfiltre er i bruk. Uten maskinlæring måtte man gå gjennom meldingene manuelt eller bruke enkle regler basert på nøkkelord. Problemet med slike løsninger er at de ofte ikke klarer å følge med på hvordan spam-meldinger stadig endrer seg.

Dette prosjektet er en prototype, ikke et ferdig produkt. Målet mitt er først og fremst å lage en fungerende nettside som bruker maskinlæring og samtidig finne ut hvor godt ulike modeller klarer å plukke ut spam-meldinger.

Metrikker

For å evaluere maskinlæringsmodellene brukes fire metrikker:

- **Accuracy:** Andelen meldinger som modellen klassifiserer korrekt.
- **Precision:** Andelen meldinger som klassifiseres som spam og faktisk er spam.

- **Recall:** Andelen av de faktiske spam-meldingene som modellen klarer å oppdage.
- **F1-score:** Et mål som kombinerer precision og recall.

For meg er recall spesielt viktig i dette prosjektet fordi jeg vil fange opp så mange spam-meldinger som mulig. Men det er også viktig med høy precision, slik at vanlige meldinger ikke blir feilaktig merket som spam.

Tanken er at modellen skal oppnå minst 95 % accuracy og minst 90 % recall på spam-meldinger i testsettet.

Jeg vurderer nettsiden ved å sjekke at det går an å skrive inn en melding, sende den til modellen og få et tydelig svar. Siden prosjektet er utviklet som en prototype, er hovedfokuset å evaluere modellens ytelse og nettsidens funksjonalitet, fremfor kommersiell lønnsomhet eller brukertilfredshet.

2: DATA

I dette prosjektet benyttes datasettet *SMS Spam Collection* fra UCI Machine Learning Repository. Datasettet inneholder 5 572 SMS-meldinger som er klassifisert som enten spam eller vanlige meldinger (ham). Datasettet er offentlig tilgjengelig og inneholder engelske meldinger.
Datasettet består av to kolonner: label og message. Kolonnen label inneholder den korrekte klassifiseringen av meldingen, mens message inneholder selve SMS-teksten. Siden meldingene allerede er merket, kan *supervised learning* benyttes, der modellen lærer sammenhengen mellom meldingsteksten og den korrekte klassifiseringen.

Fordeling av data

Datasettet inneholder følgende fordeling:

Klasse	Antall	Andel
Ham (vanlig melding)	4 825	86,6 %
Spam	747	13,4 %
Totalt	**5 572**	**100 %**

Datasettet er ubalansert fordi det inneholder betydelig flere vanlige meldinger enn spam-meldinger. Dette kan føre til at modellen blir bedre til å gjenkjenne vanlige

meldinger og skille dem fra spam. Derfor er det viktig å evaluere modellen med flere metrikker enn bare accuracy.

Databehandling

Dataene ble lest inn ved hjelp av Python-biblioteket pandas. Deretter ble datasettet delt inn i 80 % treningsdata og 20 % testdata ved hjelp av train_test_split fra scikit-learn. Jeg brukte parameteren stratify=y for å sikre at fordelingen mellom spam og vanlige meldinger var omtrent lik i begge datasettene. Satte også random_state=42 slik at oppdelingen kan gjenskapes.

Før meldingene kunne brukes til maskinlæring, måtte teksten konverteres til numeriske verdier. Dette ble gjort ved hjelp av *TF-IDF (Term Frequency–Inverse Document Frequency)*. TF-IDF representerer meldinger som numeriske vektorer basert på hvilke ord de inneholder og hvor informative ordene er i forhold til resten av datasettet. Vanlige ord får ofte lavere vekt, mens ord som er mer karakteristiske for enkelte meldinger kan få høyere vekt. TF-IDF ble implementert ved hjelp av TfidfVectorizer fra scikit-learn.

Jeg brukte en pipeline slik at tekstbehandlingen ble tilpasset treningsdataene og deretter brukt på testdataene. Dette bidrar til å unngå datalekkasje mellom trening og evaluering. Det ble ikke utført omfattende manuell datarensing eller skalering. TF-IDF håndterer grunnleggende tekstbehandling, blant annet konvertering til små bokstaver og tokenisering.

Personvern og etiske hensyn

Datasettet består av SMS-meldinger samlet inn til forskningsformål. Selv om datasettet er offentlig tilgjengelig, er det viktig å være oppmerksom på personvern ved bruk av meldingsdata. Nettsiden i dette prosjektet analyserer meldingene som brukeren selv skriver inn. Meldingen sendes til den lokale Flask-serveren for klassifisering, men lagres ikke permanent av applikasjonen. En annen begrensning er at modellen er trent på engelskspråklige meldinger. Den vil derfor ikke nødvendigvis fungere like godt med norske meldinger eller andre språk. Modellen kan også gjøre feil, og resultatene bør derfor betraktes som veiledende, ikke som en garanti for at en melding er trygg.

3: MODELLERING

Valg av modeller

I dette prosjektet har jeg undersøkt to forskjellige maskinlæringsmodeller for klassifisering av SMS-meldinger: Multinomial Naive Bayes og Logistic Regression. Begge modellene ble implementert ved hjelp av Python-biblioteket scikit-learn. Jeg valgte først Multinomial Naive Bayes fordi algoritmen er enkel, rask og ofte fungerer godt for tekstklassifisering. Modellen bruker sannsynligheter til å avgjøre om en melding tilhører klassen spam eller ham.

Etter å ha evaluert den første modellen valgte jeg å teste Logistic Regression for å undersøke om det var mulig å forbedre resultatene. Logistic Regression er en klassifikasjonsalgoritme som lærer sammenhenger mellom tekstens numeriske egenskaper og hvilken klasse meldingen tilhører. For Logistic Regression brukte jeg parameteren class_weight="balanced". Dette ble gjort fordi datasettet inneholder betydelig flere vanlige meldinger enn spam-meldinger. Parameteren gir større vekt til den underrepresenterte spamklassen under treningen.

Trening og baseline

Begge modellene ble trent på 80 % av datasettet og evaluert på de resterende 20 %. Det samme trenings- og testsettet ble brukt for begge modellene, slik at resultatene kunne sammenlignes. Teksten ble konvertert til numeriske verdier ved hjelp av TF-IDF. Dette ble implementert gjennom en pipeline i scikit-learn, der TF-IDF-behandlingen og klassifikasjonsmodellen ble kombinert.

Som en enkel baseline kan man bruke en modell som alltid klassifiserer meldinger som ham. Siden 966 av de 1 115 meldingene i testsettet er vanlige meldinger, ville en slik modell oppnådd omtrent 86,6 % accuracy. Den ville imidlertid ikke oppdaget noen spam-meldinger og dermed hatt 0 % recall for spam-klassen. Denne sammenligningen viser hvorfor accuracy alene ikke er tilstrekkelig for å evaluere en spam-detektor.

Resultater

Modellene ble evaluert ved hjelp av accuracy, precision, recall og F1-score. Precision, recall og F1-score ble beregnet med spam som positiv klasse.

Tabell 1: Sammenligning av modellresultater

Metrikk	Naive Bayes	Logistic Regression
Accuracy	96,05 %	98,39 %
Precision	100,00 %	95,17 %
Recall	70,47 %	92,62 %
F1-score	82,68 %	93,88 %

Resultatene viser at Logistic Regression oppnådde høyere accuracy, recall og F1-score enn Naive Bayes. Naive Bayes hadde derimot høyere precision. Naive Bayes klassifiserte alle de 966 vanlige meldingene riktig, men klarte bare å oppdage 105 av 149 spam-meldingene. Det betyr at 44 spam-meldinger feilaktig ble klassifisert som vanlige. Logistic Regression oppdaget 138 av 149 spam-meldinger, og overså dermed bare 11. Samtidig ble syv vanlige meldinger feilaktig klassifisert som spam.

Tabell 2: Feilklassifiseringer

Resultat	Naive Bayes	Logistic Regression
Korrekt klassifisert ham	966	959
Ham feilklassifisert som spam	0	7
Korrekt klassifisert spam	105	138
Spam feilklassifisert som ham	44	11

Ved å bruke Logistic Regression ble antallet oversette spam-meldinger redusert fra 44 til 11. Dette tilsvarer en reduksjon på 75 % sammenlignet med Naive Bayes.

Valg av endelig modell

Basert på resultatene valgte jeg Logistic Regression som den endelige modellen for nettsiden. Hovedgrunnen til dette valget er at modellen oppnådde betydelig høyere recall og F1-score. For en spam-detektor er det viktig å oppdage flest mulig uønskede meldinger, samtidig som antallet vanlige meldinger som feilaktig

klassifiseres som spam holdes lavt. Selv om Logistic Regression feilklassifiserte syv vanlige meldinger som spam, vurderte jeg forbedringen i spam-deteksjon som viktigere for denne prototypen.

Begrensninger og mulige forbedringer

Selv om Logistic Regression oppnådde gode resultater, har modellen enkelte begrensninger. Datasettet inneholder bare engelskspråklige meldinger, og modellen er derfor ikke nødvendigvis egnet til å klassifisere norske SMS-er. Modellen er også trent på et begrenset datasett, og resultatene kan være annerledes for nye meldinger med andre formuleringer. En mulig forbedring ville vært å undersøke hvilke meldinger modellen klassifiserte feil, for å identifisere mønstre i feilprediksjonene. Det kunne også vært interessant å teste andre verdier for modellens parametere eller benytte et større og nyere datasett.

Det ble ikke gjennomført en systematisk analyse av feature importance eller individuelle feilprediksjoner i denne versjonen av prosjektet. For Logistic Regression kunne modellens koeffisienter blitt undersøkt for å identifisere hvilke ord som hadde størst betydning for klassifiseringen som spam eller ham. Resultatene viser likevel at Logistic Regression er et godt valg for prototypen, med 98,39 % accuracy og 92,62 % recall på testdatasettet.

4: DEPLOYMENT

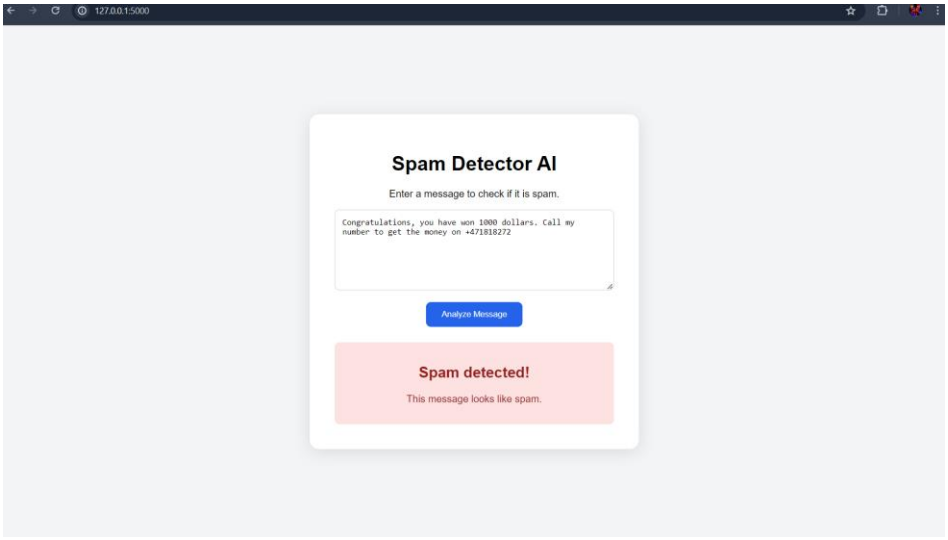
Implementering av nettsiden

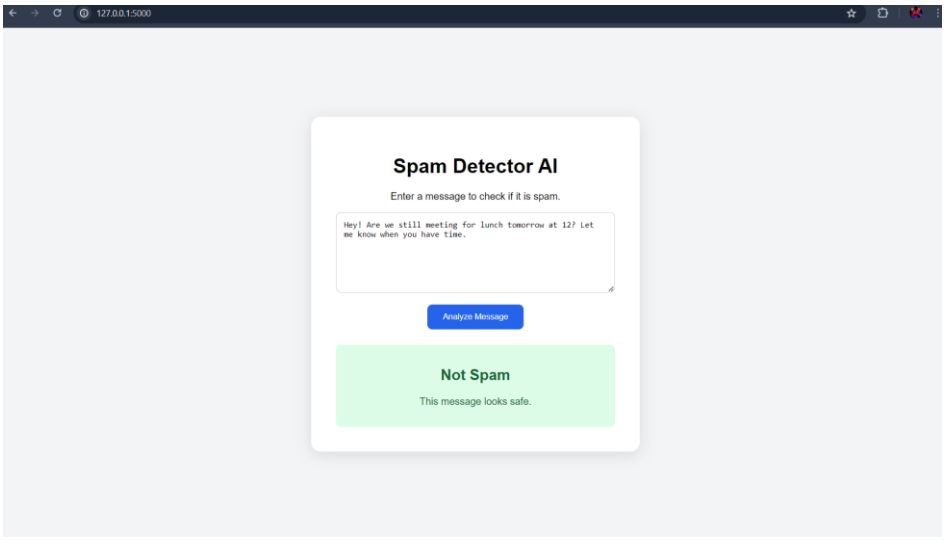
For å gjøre maskinlæringsmodellen tilgjengelig gjennom et brukergrensesnitt utviklet jeg en enkel nettside ved hjelp av Flask, HTML og CSS. Flask er et Python-basert webutviklingsrammeverk som gjør det mulig å koble en maskinlæringsmodell til en nettside. Modellen ble først trent og lagret som en joblib-fil ved hjelp av biblioteket joblib. Deretter ble den lastet inn i Flask-applikasjonen slik at den kunne brukes til å klassifisere nye meldinger. Jeg valgte Logistic Regression som den endelige modellen fordi den oppnådde best resultater under evalueringen, spesielt når det gjelder recall og F1-score.

Nettsiden har et enkelt brukergrensesnitt hvor brukeren kan skrive inn en SMS-melding og trykke på knappen «Analyze Message». Meldingen sendes deretter til Flask-serveren, hvor den ferdigtrente modellen analyserer teksten og klassifiserer den som enten spam eller ham. Resultatet vises direkte på nettsiden. Dersom meldingen klassifiseres som spam, vises en rød resultatboks med teksten «Spam detected!». Dersom meldingen klassifiseres som ham, vises en grønn resultatboks med teksten «Not Spam».

Testing og tilgjengelighet

Nettsiden ble testet lokalt ved hjelp av Flask sin innebygde utviklingsserver. Applikasjonen kan startes med kommandoen *python app.py* og åpnes i nettleseren på adressen http://127.0.0.1:5000. Det ble bekreftet at nettsiden startet og kunne kommunisere med maskinlæringsmodellen. Applikasjonen er foreløpig bare tilgjengelig lokalt og er ikke publisert på en ekstern server.





Monitorering og vedlikehold

Dersom nettsiden skulle bli tatt i bruk av faktiske brukere, ville det vært nødvendig å overvåke modellens ytelse over tid. Spam-meldinger endrer seg stadig, og modellen kan derfor bli mindre nøyaktig dersom nye typer spam skiller seg fra meldingene i treningsdatasettet. En mulig løsning ville vært å samle inn nye, korrekt merkede meldinger og trene modellen på nytt med jevne mellomrom.

Det ville også vært viktig å overvåke antall feilklassifiseringer og sørge for at vanlige meldinger ikke feilaktig blir klassifisert som spam. Ved offentlig publisering ville det i tillegg vært nødvendig å bruke en egnet produksjonsserver, sikre kommunikasjonen med HTTPS og vurdere personvernet for brukerne.

Videre utvikling

En mulig videreutvikling av prosjektet ville vært å støtte flere språk, spesielt norsk, siden modellen foreløpig er trent på engelske SMS-meldinger. Det kunne også vært nyttig å vise en sannsynlighets- eller sikkerhetsscore for prediksjonen, slik at brukeren får mer informasjon enn bare klassifiseringen som spam eller ikke-spam. En annen mulighet ville vært å legge til en funksjon der brukeren kan gi tilbakemelding dersom modellen klassifiserer en melding feil. Slike tilbakemeldinger kan senere brukes til å forbedre modellen.

5: REFERANSER

- Almeida, T. A., & Hidalgo, J. M. G. (2011). *SMS Spam Collection* [Datasett]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84
- Scikit-learn developers. (u.å.). *Scikit-learn: Machine learning in Python*. https://scikit-learn.org/stable/
- Pallets Projects. (u.å.). *Flask documentation*. https://flask.palletsprojects.com/
- pandas development team. (u.å.). *pandas documentation*. https://pandas.pydata.org/docs/
- Joblib developers. (u.å.). *Joblib documentation*. https://joblib.readthedocs.io/
- OpenAI. (2026). *ChatGPT* [KI-verktøy]. https://chatgpt.com/